AlgoBOWL: Akari

The Japanese puzzle game *Akari*, also commonly called *Light Up*, is played on a two-dimensional rectangular grid consisting of blank cells and gray cells. Light bulbs are placed in blank cells in the grid. A light bulb illuminates its own cell and all the cells in its row or column unless blocked by gray cells.

The goal of the game is to light up every blank cell on the board. However, there are two restrictions on the placement of light bulbs:

R1 A light bulb may not illuminate another light bulb.

R2 A gray cell g may contain a number between 0 and 4. This gives the exact number of light bulbs that must be placed in the up to four cells with which g shares an *edge*.

A web version of this game can be found here:

https://www.chiark.greenend.org.uk/~sgtatham/puzzles/js/lightup.html

You are encouraged to play a few rounds of this game to gain an understanding of the rules.

Your Task

Implement a solver for Akari. You are to be given inputs *which may not be fully solvable*, and are to create a solution that *illuminates all blank cells* while trying to minimize the *number of violations* of the two restrictions above. A violation results when either (i) a light bulb illuminates another light bulb (violation of **R1**) or (ii) a numbered gray cell does not have the correct number of adjacent bulbs (violation of **R2**).

Input Format

The first line of the input file contains two positive integers separated by a single space, r and c, indicating respectively the number of rows and columns in the grid.

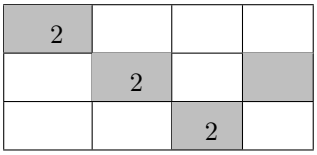
The next r lines each contain a row of the puzzle. Each line should contain exactly c characters. The characters used are:

- `.`: Blank cell.
- `X`: Gray cell.
- `0`: Gray cell marked with "0".
- `1`: Gray cell marked with "1".
- `2`: Gray cell marked with "2".
- `3`: Gray cell marked with "3".
- `4`: Gray cell marked with "4".

For example, the input

```
3 4
2...
.2.X
..2.
```

corresponds to the grid



Input Restrictions

$1 \leq r \times c \leq 10^5$

Output Format

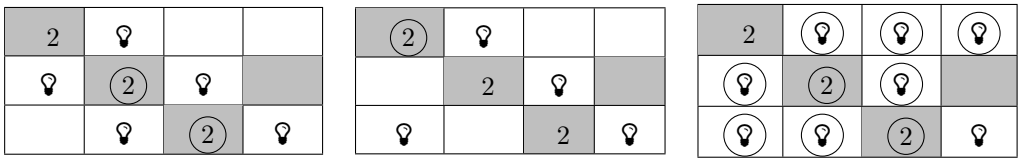
The first line of output should contain a single integer which is *the number of violations* as described earlier.

The next r lines should each contain exactly c characters. The same characters are used from the input file, with the character `L` replacing `.` characters where light bulbs were added.

Here are three different example outputs for the provided input:

```
2        1        9
2L..     2L..     2LLL
L2LX     .2LX     L2LX
.L2L     L.2L     LL2L
```

corresponding to the grids shown below. Violations are circled in each case.



All of these are *valid* outputs (because all blank cells are illuminated). Among the three, the center output is the best because it has the fewest number of violations (just 1) relative to the other two (which have 2 and 9 violations). We emphasize that an output does not need to be optimal to be valid.